

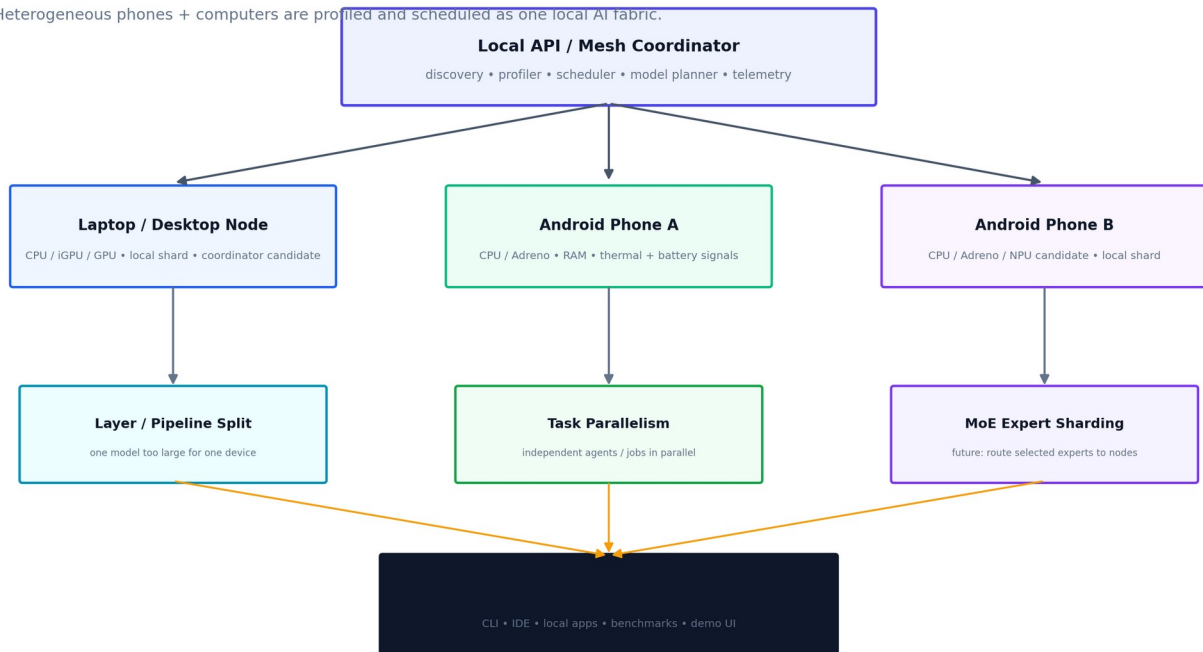
MeshAI

Technical Architecture & Feasibility Specification

Heterogeneous local AI compute across Android phones, laptops and desktops

MeshAI: Proposed Runtime Architecture

Heterogeneous phones + computers are promoted and scheduled as one local AI fabric.



SCOPE DISCIPLINE: This document separates upstream capabilities already demonstrated by existing projects from the components MeshAI proposes to engineer. Performance targets are not presented as facts until measured on the actual hardware.

Prepared for hackathon prototype design and technical review • September 2026

01 — DEFINITION

What MeshAI is — and is not

A local orchestration layer that treats nearby consumer devices as a heterogeneous AI fabric.

MeshAI is a proposed local-first runtime that discovers nearby computers and Android phones, profiles their usable memory and compute capability, measures the interconnect, and chooses an execution strategy for an AI workload. The goal is not to beat cloud datacenters on raw latency. The goal is to make fragmented hardware useful as one private, offline-capable system when a single device is capacity-constrained or when multiple independent jobs can be parallelized.

Question	Technical answer
What problem is solved?	Useful compute and memory are fragmented across devices; local AI is usually bounded by one machine.
What is the primary prototype?	Laptop/desktop coordinator + one or more Android workers on a trusted local network.
What is reused?	llama.cpp / GGML backends, GGUF models, local RPC concepts, existing Android acceleration paths.
What is new engineering?	Cross-device product layer: discovery, profiling, strategy selection, mobile-aware scheduling, orchestration, observability and safe local API exposure.
What is deliberately not claimed?	Novel invention of distributed transformers; guaranteed speedups; guaranteed 14B→32B without hardware benchmarks.

DESIGN POSITION: The prototype should be judged on measured capability: model fit, context capacity, latency, throughput, thermals, and reliability — not on parameter-count marketing.

02 — EXISTING FOUNDATIONS

What already exists upstream

The project is viable because the difficult primitives are partially available; the gap is heterogeneous consumer orchestration.

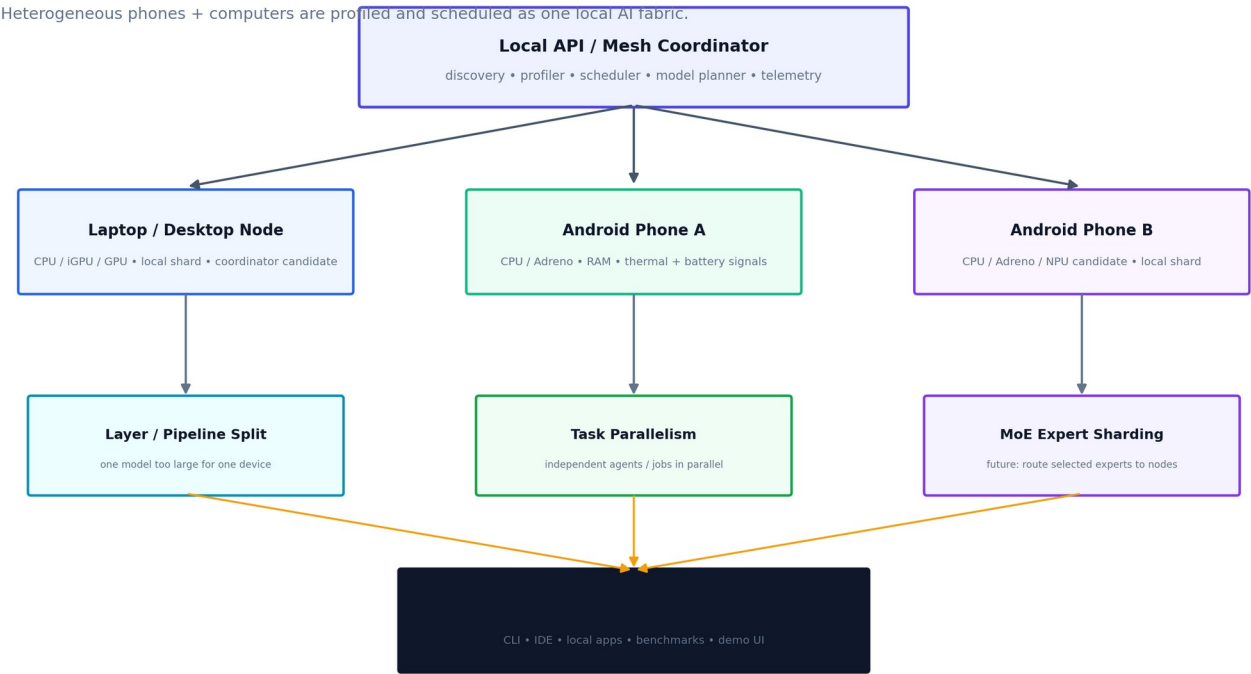
System / project	Verified capability	Implication for MeshAI
llama.cpp RPC	Remote hosts can expose GGML accelerator devices; one client can offload computation to one or more RPC servers. RPC is explicitly proof-of-concept and insecure on open networks.	Use as MVP primitive on a trusted LAN; add pairing/security later.
llama.cpp Android	Official Android binding and NDK build paths exist.	Android worker is practical without inventing an inference engine.
llama.cpp OpenCL	Qualcomm Adreno backend supports Android; Adreno 840 is listed as supported.	Modern Snapdragon phones can be tested as actual GPU workers, not just RAM stores.
llama.cpp Hexagon	Snapdragon/Hexagon backend exists; active development continues.	NPU acceleration is an advanced target; distributed RPC→Hexagon must be experimentally validated.
Exo	Automatic discovery, topology-aware partitioning and API compatibility across supported systems.	UX/scheduler benchmark; shows product-level orchestration is valuable.
AI Doomsday Toolbox	Android master/worker distributed inference with RAM-aware layer splitting.	Proves Android phone clusters are real; our scope must go beyond Android-only/manual flows.
Synapse	Cross-device browser/WebGPU layer sharding across phones/laptops; published examples include ~1.3 tok/s Pixel→iPhone and ~0.82 tok/s on two Qualcomm Android phones.	Proof that heterogeneous boundaries work; also proves latency is a real bottleneck.
Meshlit	Android federated inference; favors data-parallel jobs and is developing MoE routing.	Supports AUTO-mode strategy: do not force layer splitting for every workload.

03 — RUNTIME ARCHITECTURE

Proposed system architecture

A thin local control plane sits above existing runtimes and decides how to execute each workload.

MeshAI: Proposed Runtime Architecture



Component	Responsibility
Mesh Coordinator	Owns cluster state, discovery, model plan, strategy selection, session routing, endpoint exposure and telemetry.
Device Agent	Reports hardware, RAM, backends, battery/charging, thermal headroom, network measurements and worker health.
Inference Worker	Runs a local backend (initially llama.cpp CPU/OpenCL; Hexagon later) and owns its assigned model shard/state.
Model Planner	Maps model bytes, layer tensors, context/KV requirements and runtime buffers onto available memory.
Scheduler	Selects single-node, layer split, task parallel, or future MoE mode; may reject slow/hot/low-battery nodes.
Local API	Presents one stable localhost endpoint so IDEs, CLIs and apps do not need to know the physical topology.

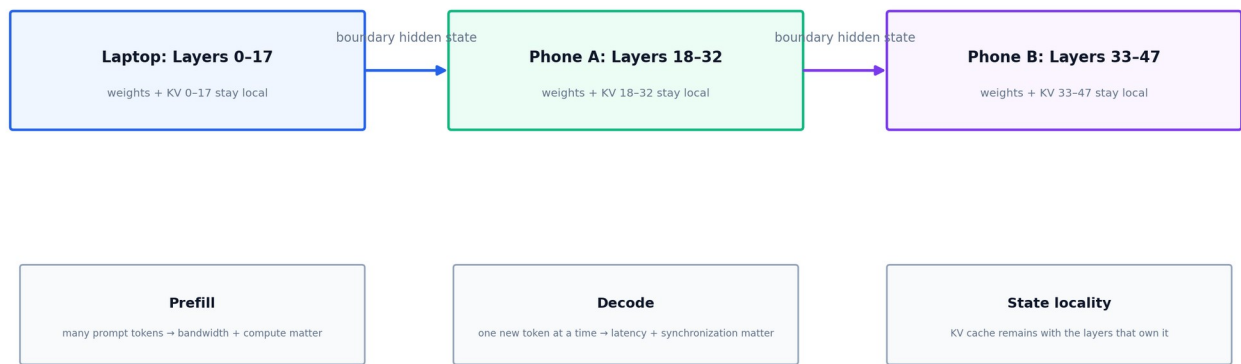
04 — PIPELINE/LAYER SPLITTING

How one model can span multiple devices

Contiguous transformer blocks are kept on different nodes. We minimize cross-device boundaries.

Layer split: what moves, and what stays resident

The model is not re-sent for every token. Shards remain resident; only boundary activations and control traffic cross nodes.



For a sequential transformer, layer n+1 requires the hidden state produced by layer n. Therefore, a simple distributed deployment is a pipeline: early layers on one node, middle layers on another, final layers on another. Model weights remain resident on the node that owns them; each node also keeps the KV cache for its own attention layers. Boundary activations move forward between devices.

Property	Design consequence
Contiguous layer blocks	Avoid alternating devices layer-by-layer; that would create a network hop after every transformer block.
Weights stay resident	Do not resend model weights for every token. Cache or pre-stage model shards locally.
KV stays with layer owner	Conversation state does not need to bounce entirely between nodes on each token.
Prefill ≠ decode	Prompt ingestion is bandwidth/compute heavy; token-by-token decode is more sensitive to RTT and synchronization.
Capacity can improve without speed improving	A mesh can make an otherwise-unloadable model possible while still being slower than a smaller single-node model.

05 — WHAT CROSSES THE NETWORK

Boundary data, KV cache and an illustrative size calculation

The network does not carry the entire model per token; it carries intermediate state and protocol traffic.

Example reference model: Qwen3-Coder-30B-A3B-Instruct reports 48 hidden layers, hidden size 2048, 4 KV heads, head dimension 128, 128 experts and 8 experts selected per token. These values make the dataflow concrete, but MeshAI should remain model-agnostic.

Quantity	Illustrative calculation	Interpretation
Decode boundary hidden state	$2048 \text{ values} \times 2 \text{ bytes (FP16/BF16)} \approx 4 \text{ KB}$	Conceptual payload per sequence element at a layer-group boundary; real RPC traffic is larger because of batching, metadata and synchronization.
4K-token prefill boundary	$4096 \times 2048 \times 2 \text{ bytes} \approx 16 \text{ MB}$	Prompt processing can move much larger tensors than single-token decode.
16K-token prefill boundary	$16384 \times 2048 \times 2 \text{ bytes} \approx 64 \text{ MB}$	Bandwidth becomes more important as prompt batches grow.
KV per token, all 48 layers	$48 \times 2(K,V) \times 4 \text{ heads} \times 128 \times 2 \text{ bytes} \approx 96 \text{ KiB/token}$	Approximate FP16 KV footprint before runtime-specific optimizations/quantization.
KV at 16K context	$\approx 1.5 \text{ GiB}$	Context can consume meaningful pooled memory even when weights already fit.

DESIGN POSITION: These are architecture-level estimates, not benchmark results. The actual runtime can use different dtypes, batching, layouts, quantized KV and additional buffers.

06 — PARALLELISM MODES

Why the scheduler needs more than one strategy

Layer splitting solves capacity. Task parallelism solves throughput. Tensor parallelism is interconnect-sensitive. MoE adds another option.

Mode	Execution model	Best use	Mobile/LAN position
Single-node	One device loads and runs model	Model already fits and one node is fastest	Preferred when adequate; simplest and lowest synchronization cost.
Layer / pipeline	Contiguous layer groups live on different nodes	One dense model cannot fit on one device	Primary MVP distributed mode.
Task / data parallel	Independent jobs or agents run on separate nodes	Coding agents, batch embeddings, image variants, evals	Very strong on phones because nodes rarely exchange intermediate tensors.
Tensor parallel	Every node computes part of the same layer	Fast multi-GPU systems with high-bandwidth links	Not a Wi-Fi-first strategy; keep experimental.
MoE expert sharding	Different experts reside on different nodes; router invokes selected experts	Sparse MoE models where only a subset of experts fire per token	Promising future direction; requires dedicated routing/state design.

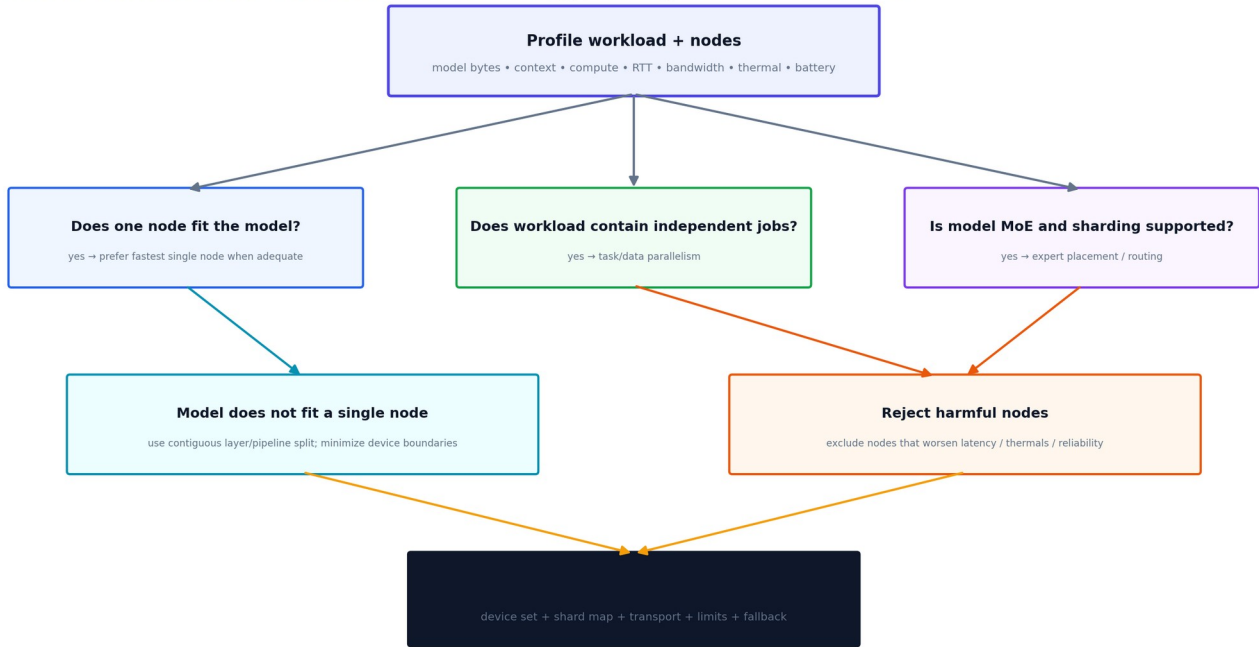
For coding, MeshAI can expose two useful behaviors: DEEP mode pools devices when a stronger model or larger context is required; FAST mode gives separate agents/jobs to independent nodes. AUTO mode chooses based on measured topology and workload structure.

07 — AUTO SCHEDULER

The main product-level technical contribution

A good mesh must know when adding a device helps — and when it makes every request worse.

AUTO Scheduler: choose the strategy, not just the devices



Signal	How it should influence placement
Usable RAM / accelerator memory	Hard capacity constraint. Reserve OS/runtime headroom rather than using nominal device RAM.
Per-layer / token benchmark	Estimate actual node compute time for the selected backend and quantization.
RTT	Critical for sequential boundaries during decode. High RTT can make a theoretically powerful phone harmful.
Bandwidth	Critical for prefill, shard loading and larger activation batches.
Thermal headroom	Reduce/reject node before sustained throttling destroys predictable latency.
Battery + charging	Prefer charging devices; respect user-defined minimum battery and power limits.
Reliability / loss	Avoid nodes with unstable links or repeated worker failures.

DESIGN POSITION: First implementation can use a heuristic score. A later version can learn a cost model from measured per-layer latency and transfer time, then solve for the lowest-latency feasible placement.

08 — ANDROID WORKER DESIGN

What runs on the phone

The phone must be a real compute node with transparent resource controls, not a remote display.

Layer	Proposed implementation
UI	Native Android/Jetpack Compose or Flutter shell with clear Join/Leave, model cache, battery/thermal limits and live telemetry.
Native runtime	NDK-packaged llama.cpp / GGML worker; initial CPU backend, then Qualcomm Adreno OpenCL.
RPC/worker service	Foreground service while participating in a mesh; trusted-LAN only in MVP.
Device profiler	Memory headroom, SoC/backend inventory, benchmark score, Android thermal status/headroom, battery/charging.
Model cache	Store assigned model files/shards locally; validate hashes; avoid transferring multi-GB tensors on every session.
Network stack	MVP: same-LAN TCP. Product path: local discovery, Wi-Fi Direct/NSD, possibly USB network transport.
NPU path	Hexagon is an advanced target. Verify standalone backend first, then test whether the RPC worker can reliably expose/use it.

The upstream llama.cpp OpenCL backend explicitly targets Qualcomm Adreno and currently lists Android support for recent Snapdragon devices, including Adreno 840. This makes GPU-backed Android workers a concrete experiment rather than a speculative feature.

09 — DESKTOP/COORDINATOR DESIGN

What runs on the laptop or desktop

The coordinator should remain useful even if the cluster consists of only one machine.

Subsystem	Behavior
Discovery	Find approved workers on LAN; require explicit pair/accept before compute is exposed.
Capability inventory	List local and remote CPU/GPU/NPU backends; normalize memory/compute/network measurements.
Model catalog	Parse GGUF metadata; estimate weight bytes, runtime headroom and context/KV requirements.
Planner	Generate feasible placements for the available node set; prefer fewer boundaries for pipeline mode.
Execution	Launch/attach worker processes and expose one localhost API endpoint.
Observability	Per-node RAM, backend, temperature, battery, RTT, bandwidth, shard map, TTFT, prompt tok/s, decode tok/s.
Fallback	If a remote node disappears, stop/pause safely, rebuild a feasible placement, reload state/context where possible, then resume a new generation.

DESIGN POSITION: Instant live migration mid-token is not an MVP requirement. Honest “pause → rebalance → restore context → resume” behavior is technically credible and demo-safe.

10 — NETWORKING

Transport decisions and why boundaries matter

The scheduler must model the interconnect as a first-class resource.

Transport	MVP / product role	Trade-off
Same Wi-Fi LAN	MVP default	Easy to implement and debug; performance depends heavily on router/channel/RTT.
Wi-Fi Direct	Product improvement	Direct peer connectivity without an access point; useful for portable phone clusters.
USB network / tethered link	Performance experiment	Potentially lower jitter/stable throughput, but device/driver UX is harder.
Ethernet / USB-Ethernet	Enthusiast / lab path	Stable wired bandwidth; less consumer-friendly.
WAN / Internet	Not for dense layer splitting	Sequential activation hops over wide-area links usually make latency unacceptable; use independent jobs instead.

Networking creates two distinct optimization problems: prefill benefits from high throughput because batched prompt activations are larger; decode often benefits more from low round-trip time and low synchronization overhead because generation is sequential. The scheduler should therefore benchmark both throughput and RTT rather than using “Wi-Fi speed” as one number.

11 — SECURITY AND TRUST

Why raw RPC cannot be the product boundary

The upstream RPC mechanism is useful for a prototype but is explicitly not safe for open networks.

Threat	MVP mitigation	Product direction
Unauthenticated worker access	Private trusted LAN; explicit device approval; no port exposure to Internet	Mutual device identity + encrypted authenticated transport (e.g. mTLS/Noise-style session).
Malicious model/shard	Use known model hashes and signed manifests	Content-addressed cache + signature verification + provenance metadata.
Arbitrary remote code	Do not accept arbitrary binaries/jobs in local-mesh mode	Fixed worker runtime and constrained workload schema.
Sensitive prompts/code	Keep data on paired local devices	User-visible topology, explicit permissions, encrypted links and audit logs.
Compute rental abuse	Out of MVP	Sandboxed predefined workloads, quotas, verification, payment/reputation and abuse handling.

DESIGN POSITION: Earn/marketplace mode should be architecturally separate from local mesh mode. Remote strangers should receive coarse independent jobs, not become latency-critical transformer stages.

12 — BENCHMARK AND KILL-TEST PLAN

What must be measured before the pitch makes hard claims

The first experiment is intentionally small; it answers whether the architecture is worth scaling.

Phase	Topology	Model / task	Success criterion
A — Baseline	Laptop only	1.5B/3B GGUF	Record load time, RAM, TTFT, prompt tok/s, decode tok/s.
B — 1 phone CPU	Laptop + Android	Same model, contiguous layer split	Logs prove remote layers execute; inference is correct and stable.
C — 1 phone GPU	Laptop + Android Adreno	Same split	Measure whether GPU backend offsets network boundary cost.
D — 2 phones	Laptop + two Androids	3B/7B then larger	Stable shard placement across 3 nodes; record incremental capacity and latency.
E — Capacity test	Best topology	14B / 30B-class quantized models	Find the largest practical model/context that fits and remains usable.
F — Parallel test	3 nodes	Independent coding/review/eval tasks	Compare total jobs/min against single-node sequential execution.

Mandatory metric	Reason
Usable memory per node	Proves capacity instead of quoting nominal RAM.
TTFT	Captures load/prefill/network delay perceived by user.
Prompt tok/s	Measures prefill and bandwidth-sensitive behavior.
Decode tok/s	Measures interactive generation quality.
Network throughput + RTT	Explains why a device helps or hurts.
Temperature / thermal status	Shows mobile sustainability.
Battery / power state	Supports mobile-aware scheduling.
Model + quantization + context	Makes every result reproducible.

13 — CODING WORKLOAD

Why CLI development is a credible demonstration

The mesh should expose compute, not force users into a new chatbot UI.

The coordinator can expose a localhost OpenAI-compatible endpoint. Existing CLIs/IDEs can point to that endpoint, while MeshAI remains responsible for topology and execution. A coding demonstration is useful because it exercises context, tool loops and repeated inference rather than a single toy prompt.

Mode	Example topology	Coding behavior
DEEP	Laptop + phones pool for one larger model	Architecture/debugging task uses one stronger local model or higher-quality quantization/context.
FAST	Laptop runs main agent; Phone A reviewer; Phone B test/log analyzer	Independent tasks execute concurrently; coordinator merges results.
AUTO	Scheduler chooses based on model fit, task graph and network	Use single node for simple tasks; pool only when capacity is needed; parallelize independent jobs.

DESIGN POSITION: The claim is not “local mesh beats frontier cloud coding.” The claim is “useful private/offline coding becomes possible on hardware already owned, with the scheduler choosing the least harmful execution mode.”

14 — NOVELTY BOUNDARY

What MeshAI can honestly claim

A mature pitch acknowledges adjacent systems and states the specific integration gap.

Already demonstrated elsewhere	MeshAI focus
Distributed model inference	Heterogeneous consumer-device orchestration.
Android local inference	Zero-Termux / zero-IP-address worker UX.
Android phone clustering	Android + laptop/desktop topology as one product.
Automatic partitioning on some platforms	Mobile-aware scheduling using thermals, battery, RTT and backend benchmarks.
Data/task parallelism	One AUTO scheduler that can choose single-node, pipeline split or independent jobs.
MoE research/prototypes	Future integration as a strategy, not an MVP claim.
OpenAI-compatible local servers	Stable local endpoint independent of the changing physical device topology.

The strongest technical thesis is therefore not “we split layers.” It is: “we build a local control plane that continuously decides how heterogeneous phones and computers should cooperate for an AI workload — including deciding not to use a device when it is harmful.”

15 — RISKS AND ENGINEERING DECISIONS

Known failure modes before implementation

The prototype is feasible, but several areas must be treated as experiments rather than assumptions.

Risk	Impact	Mitigation / decision
RPC boundary latency dominates	Distributed model fits but feels unusably slow	Benchmark first; minimize boundaries; prefer task parallelism where appropriate.
Phone GPU/NPU backend incompatibility	CPU-only worker may be too slow	CPU baseline → Adreno OpenCL → Hexagon as staged milestones.
Android memory pressure kills worker	Unstable larger-model runs	Reserve headroom; smaller context; controlled worker service; cache carefully.
Thermal throttling	Performance decays during demo	Thermal-aware admission; charging/cooling; cap per-device load.
Node disconnect	Generation fails	MVP pause/replan/restart with preserved prompt/context where practical.
Model transfer time	Cluster startup takes minutes	Persistent local cache, hashes and pre-staged demo models.
Competitor overlap	Novelty challenge	Be explicit about reused primitives; emphasize heterogeneous scheduler/product integration.
Marketplace scope creep	Destroys hackathon focus	Keep earn mode roadmap-only.

16 — RECOMMENDED MVP SCOPE

What to build for the hackathon

A small, measurable distributed runtime beats a broad platform that only exists in slides.

- Laptop coordinator + Android worker app on the same trusted Wi-Fi network.
- One real distributed LLM path using contiguous layer splitting through llama.cpp/GGML RPC or equivalent native worker plumbing.
- Device profiling: usable RAM, backend, CPU/GPU benchmark, RTT, bandwidth, thermal status/headroom, battery and charging state.
- AUTO planner that can select single-node vs layer split vs task-parallel for a small set of supported workloads.
- Live dashboard showing node health, model placement, shard/layer map, TTFT and token throughput.
- One localhost API endpoint usable by a simple CLI or coding tool.
- Benchmark report: laptop only vs +1 phone vs +2 phones with all model/quantization/context details.
- No global marketplace, no unverified instant hot migration, no dependence on Hexagon NPU for MVP success.

DESIGN POSITION: Demo success condition: judges can see that the phones are executing real work, understand why the scheduler chose the topology, and observe a measurable capability/throughput benefit with the Internet disconnected.

REFERENCES

Primary technical sources used in this specification

URLs are included for engineering follow-up. Accessed September 2026.

1. llama.cpp RPC backend

<https://github.com/ggml-org/llama.cpp/blob/master/tools/rpc/README.md>

2. llama.cpp Android build/binding

<https://github.com/ggml-org/llama.cpp/blob/master/docs/android.md>

3. llama.cpp OpenCL / Qualcomm Adreno backend

<https://github.com/ggml-org/llama.cpp/blob/master/docs/backend/OPENCL.md>

4. llama.cpp build / backend matrix

<https://github.com/ggml-org/llama.cpp/blob/master/docs/build.md>

5. llama.cpp Hexagon / FastRPC development discussion

<https://github.com/ggml-org/llama.cpp/discussions/26227>

6. Exo

<https://github.com/exo-explore/exo>

7. AI Doomsday Toolbox

<https://github.com/ManuXD32/AI-Doomsday-Toolbox>

8. Synapse distributed WebGPU inference

<https://github.com/tejasphatak/Synapse>

9. Meshlit Android federated inference

<https://github.com/sabbirimon/meshlit>

10. Qwen3-Coder-30B-A3B-Instruct configuration

<https://huggingface.co/Qwen/Qwen3-Coder-30B-A3B-Instruct/blob/main/config.json>

End of technical specification